

Ex.No:17

Experiment : Write a Prolog Program to Sum the Integers from 1 to n

Aim

To write a Prolog program to find the sum of integers from 1 to n.

Objectives

- To understand recursion in Prolog.
- To calculate the sum of numbers from 1 to n.
- To display the result.

Algorithm

1. Start.
2. Read the value of n.
3. If $n = 0$, return 0.
4. Otherwise, add n to the sum of $n-1$.
5. Display the result.
6. Stop.

Program

sum(0,0).

sum(N,S):-

$N > 0$,

 N1 is N-1,

 sum(N1,S1),

 S is N+S1.

Sample Input

?- sum(5,S).

Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/sum.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/sum.pl compiled, 5 lines read - 826 bytes written, 8 ms
| ?- sum(5, S).

S = 15 ?

yes
| ?- |
```

Result

Thus, the Prolog program to find the sum of integers from 1 to n was implemented successfully.

Ex.No:18

Experiment : Write a Prolog Program for Name and DOB Database

Aim

To write a Prolog program to create a database containing Name and Date of Birth (DOB).

Objectives

- To understand Prolog facts.
- To create a simple database.
- To retrieve the stored information.

Algorithm

1. Start.
2. Store Name and DOB as facts.
3. Enter a query.
4. Display the matching record.
5. Stop.

Program

```
person(ram,'10-01-2002').
```

```
person(sita,'15-08-2003').
```

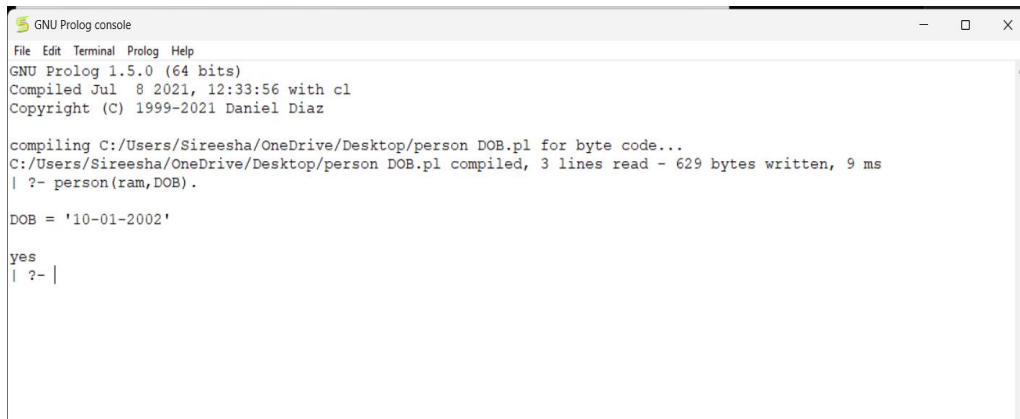
```
person(ravi,'25-12-2001').
```

```
person(priya,'05-06-2004').
```

Sample Input

```
?- person(ram,DOB).
```

Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/person DOB.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/person DOB.pl compiled, 3 lines read - 629 bytes written, 9 ms
| ?- person(ram,DOB).

DOB = '10-01-2002'

yes
| ?- |
```

Result

Thus, the Prolog program to create a Name and DOB database was implemented successfully.

Ex.No:19

Experiment : Write a Prolog Program for Student–Teacher–Subject Code Database

Aim

To write a Prolog program to create a database containing Student Name, Teacher Name, and Subject Code.

Objectives

- To understand Prolog databases.
- To store student details.
- To retrieve teacher and subject information.

Algorithm

1. Start.
2. Store Student, Teacher, and Subject Code.
3. Enter a query.
4. Display the result.
5. Stop.

Program

```
student_teacher(ram,ramesh,cs101).
```

```
student_teacher(sita,suresh,ai102).
```

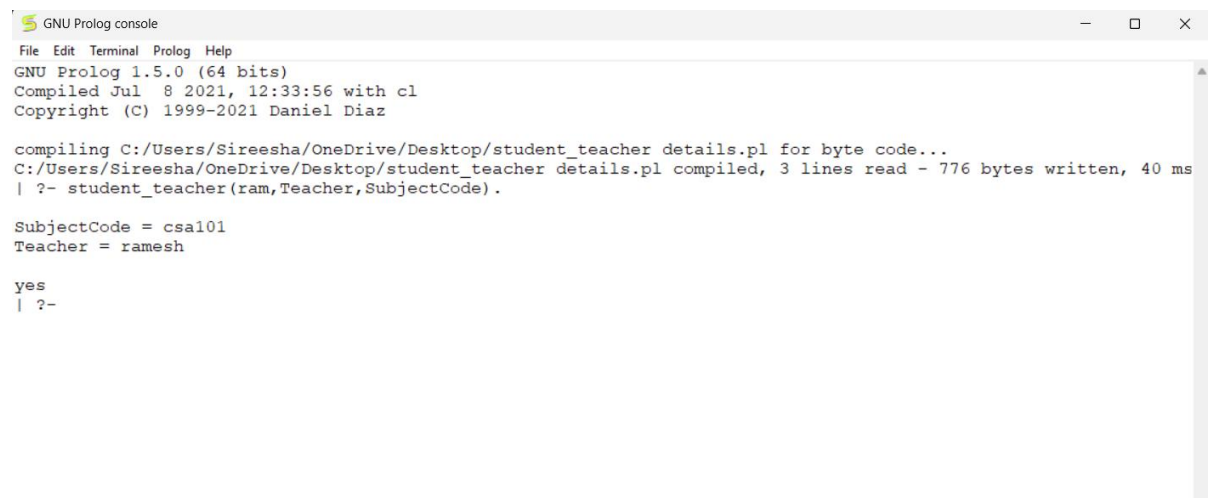
```
student_teacher(ravi,mahesh,db103).
```

```
student_teacher(priya,anitha,cn104).
```

Sample Input

```
?- student_teacher(ram,Teacher,SubjectCode).
```

Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/student_teacher details.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/student_teacher details.pl compiled, 3 lines read - 776 bytes written, 40 ms
| ?- student_teacher(ram,Teacher,SubjectCode).

SubjectCode = csal01
Teacher = ramesh

yes
| ?-
```

Result

Thus, the Prolog program for the Student–Teacher–Subject Code database was implemented successfully.

Ex.No:20

Experiment : Write a Prolog Program for Planets Database

Aim

To write a Prolog program to create a database containing the names of planets and their order from the Sun.

Objectives

- To understand Prolog facts.
- To create a planets database.
- To retrieve planet information.

Algorithm

1. Start.
2. Store planet names and positions.
3. Enter a query.
4. Display the required information.
5. Stop.

Program

```
planet(mercury,1).
```

```
planet(venus,2).
```

```
planet(earth,3).
```

```
planet(mars,4).
```

```
planet(jupiter,5).
```

```
planet(saturn,6).
```

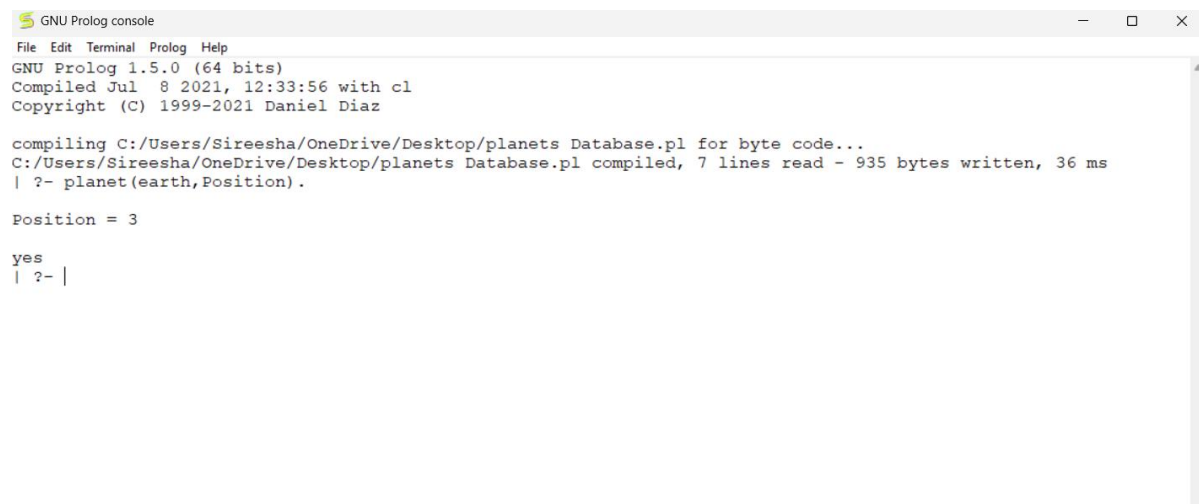
```
planet(uranus,7).
```

```
planet(neptune,8).
```

Sample Input

```
?- planet(earth,Position).
```

Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/planets Database.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/planets Database.pl compiled, 7 lines read - 935 bytes written, 36 ms
| ?- planet(earth,Position).

Position = 3

yes
| ?- |
```

Result

Thus, the Prolog program for the Planets Database was implemented successfully.

Ex.No:21

Experiment : Write a Prolog Program to Implement Towers of Hanoi

Aim

To write a Prolog program to solve the Towers of Hanoi problem.

Objectives

- To understand recursion in Prolog.
- To move all disks from source to destination.
- To follow the rules of the Towers of Hanoi.

Algorithm

1. Start.
2. Read the number of disks.
3. Move N-1 disks from source to auxiliary.
4. Move the largest disk to destination.
5. Move N-1 disks from auxiliary to destination.
6. Stop.

Program

`move(1,X,Y,_):-`

```
    write('Move disk from '),  
    write(X),  
    write(' to '),  
    write(Y),nl.
```

`move(N,X,Y,Z):-`

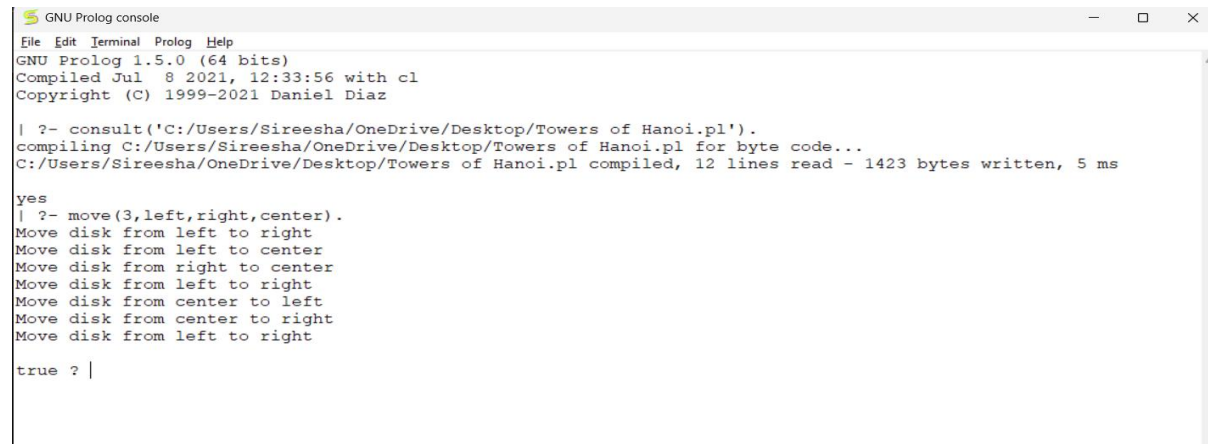
```
    N>1,  
    N1 is N-1,  
    move(N1,X,Z,Y),  
    move(1,X,Y,_),
```

move(N1,Z,Y,X).

Sample Input

?- move(3,left,right,center).

Output

A screenshot of a GNU Prolog console window. The window title is "GNU Prolog console". The menu bar includes "File", "Edit", "Terminal", "Prolog", and "Help". The console text shows the Prolog version (1.5.0, 64 bits), compilation date (Jul 8 2021), and the path to the Towers of Hanoi program. The user enters a query to consult the file and then a specific move command. The program outputs a sequence of seven moves required to solve the 3-disk Tower of Hanoi problem. The console ends with "true ? |".

```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul 8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/Sireesha/OneDrive/Desktop/Towers of Hanoi.pl').
compiling C:/Users/Sireesha/OneDrive/Desktop/Towers of Hanoi.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/Towers of Hanoi.pl compiled, 12 lines read - 1423 bytes written, 5 ms

yes
| ?- move(3,left,right,center).
Move disk from left to right
Move disk from left to center
Move disk from right to center
Move disk from left to right
Move disk from center to left
Move disk from center to right
Move disk from left to right

true ? |
```

Result

Thus, the Towers of Hanoi problem was implemented successfully using Prolog.

Ex.No:22

Experiment 22: Write a Prolog Program to Print Particular Bird Can Fly or Not

Aim

To write a Prolog program to determine whether a bird can fly.

Objectives

- To understand facts and rules.
- To identify birds that can fly.
- To display the result.

Algorithm

1. Start.
2. Store bird names.
3. Define birds that can fly.
4. Query the bird.
5. Display whether it can fly.
6. Stop.

Program

bird(parrot).

bird(crow).

bird(pigeon).

bird(peacock).

bird(ostrich).

bird(penguin).

can_fly(parrot).

can_fly(crow).

can_fly(pigeon).

can_fly(peacock).

Sample Input

?- can_fly(parrot).

Another Input

?- can_fly(ostrich).

Output

A screenshot of the GNU Prolog console window. The window has a title bar with the text "GNU Prolog console" and a status bar that says "Document was last saved: Just now". The menu bar includes "File", "Edit", "Terminal", "Prolog", and "Help". The main text area shows the following output:

```
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/Bird Can Fly or Not.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/Bird Can Fly or Not.pl compiled, 11 lines read - 930 bytes written, 5 ms
| ?- can_fly(parrot).

yes
| ?- can_fly(ostrich).

no
| ?- |
```

Result

Thus, the Prolog program to determine whether a bird can fly was implemented successfully.